Chapter 1 solutions

CH01-Q01

1. The ISA defines instruction encodings and semantics, visible registers, addressing, and exceptions. Correct implementations preserve the defined meaning of existing machine code.
2. Pipeline depth, caches, predictors, issue width, forwarding, and stalls may change. These change CPI, clock period, and timing observations.
3. Even with similar dynamic IC, CPI and frequency may differ. Since $T = IC \times CPI/f$, semantic compatibility supplies no equal-time guarantee.

CH01-Q02

P executes

$$C_P = (3.6 \times 10^8)(1.50) = 5.4 \times 10^8, \qquad T_P = \frac{5.4 \times 10^8}{2.4 \times 10^9} = 0.225\ \text{s} = 225\ \text{ms}.$$

Q executes

$$C_Q = (3.0 \times 10^8)(1.80) = 5.4 \times 10^8, \qquad T_Q = \frac{5.4 \times 10^8}{2.8 \times 10^9} = 0.192857\ \text{s}.$$

Using P as baseline,

$$S_{Q/P} = 0.225/0.192857 = 1.16667,$$
$$\frac{T_P - T_Q}{T_P} = \frac{0.225 - 0.192857}{0.225} = 14.286\%.$$

A speedup of 1.167 does not mean a 16.7% time reduction.

At CPI 2.10,

$$T'_Q = \frac{(3.0 \times 10^8)(2.10)}{2.8 \times 10^9} = 0.225\ \text{s} = T_P.$$

Its 6.3×10^8 cycles exceed P's 5.4×10^8; the higher 2.8 GHz frequency compensates. At a common 2.4 GHz, this Q would be 16.667% slower.

For the final variant,

$$T''_Q = \frac{(3.0 \times 10^8)(1.40)}{2.4 \times 10^9} = 0.175\ \text{s}, \qquad S = 0.225/0.175 = 1.28571.$$

The compiler changes IC. Instruction mix, ISA choices, and microarchitectural stalls affect CPI. Clock paths and hardware determine f. Compare all factors in the same time equation.

Chapter 1 solutions (continued)

CH01-Q03

$$IC = (2.0 + 0.5 + 0.5)10^9 = 3.0 \times 10^9,$$
$$C = [2.0(1) + 0.5(4) + 0.5(3)]10^9 = 5.5 \times 10^9,$$
$$CPI_{\text{avg}} = 5.5/3 = 1.83333, \qquad T = 5.5 \times 10^9/(3.0 \times 10^9) = 1.83333\ \text{s}.$$

Reducing load/store CPI saves 0.5×10^9 cycles:

$$C' = 5.0 \times 10^9, \quad T' = 1.66667\ \text{s}, \quad S = 5.5/5 = 1.10.$$

S1 executes $2 + 1 + 2 = 5$ instructions; S2 executes $4 + 1 + 1 = 6$.

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \quad CPI_1 = 10/5 = 2.0,$$
$$C_2 = 4(1) + 1(2) + 1(3) = 9, \quad CPI_2 = 9/6 = 1.5.$$

At a common frequency $S_{2/1} = 10/9 = 1.111$. S2 has 20% more instructions but fewer cycles per instruction and fewer total cycles. Instruction count alone wrongly selects S1. The comparison must include instruction mix and CPI.

CH01-Q04

Normalize baseline time to 1:

$$T_1 = 0.28 + 0.72/6 = 0.40, \qquad S = 1/0.40 = 2.5.$$
$$T_\infty = 0.28, \qquad S_\infty = 1/0.28 = 25/7 = 3.57143.$$

For finite positive s, $0.72/s > 0$, so $T > 0.28$ and $S < 25/7$. The bound is approached, never attained by finite local acceleration. Added costs further limit actual designs.

For a 20 s target,

$$20 = 80/s + 20 \quad \Rightarrow \quad 80/s = 0.$$

No finite s satisfies it. Infinite acceleration only approaches 20 s. For 50 s,

$$50 = 80/s + 20 \quad \Rightarrow \quad 30s = 80 \quad \Rightarrow \quad s = 8/3 = 2.667.$$

CH01-Q05

$$X_{\text{old}} = 1/(0.020\ \text{s}) = 50\ \text{frames/s}, \qquad X_{\text{pipe}} = 1/(0.008\ \text{s}) = 125\ \text{frames/s}.$$

With equally timed stages and completions on phase boundaries, frame 1 finishes at 24 ms and frame 2 one initiation interval later at 32 ms. A single frame becomes slower, from 20 to 24 ms. Multiple frames are required to realize the throughput advantage.

CH01-Q06

$$GM_X = (1.44 \cdot 1.00 \cdot 0.81)^{1/3} = 1.0527, \qquad GM_Y = (1.20 \cdot 1.25 \cdot 1.08)^{1/3} = 1.1747.$$

Chapter 1 solutions (continued)

Y has the higher equally weighted geometric mean. For arbitrary replacement reference times,

$$r^\star_{X,i} = T^\star_{\mathrm{ref},i}/T_{X,i}, \qquad r^\star_{Y,i} = T^\star_{\mathrm{ref},i}/T_{Y,i},$$

$$\frac{r^\star_{X,i}}{r^\star_{Y,i}} = \frac{T_{Y,i}}{T_{X,i}},$$

$$\frac{GM^\star_X}{GM^\star_Y} = \left(\prod_i \frac{T_{Y,i}}{T_{X,i}}\right)^{1/n} = \frac{GM_X}{GM_Y}.$$

Each reference time cancels separately; no common rescaling is needed. The ratio of arithmetic means instead contains the two sums $\sum_i T^\star_{\mathrm{ref},i}/T_{X,i}$ and $\sum_i T^\star_{\mathrm{ref},i}/T_{Y,i}$. Different per-program reference changes alter their relative weighting, so that ratio is not invariant.

For almost pure R1, X's 1.44 exceeds Y's 1.20. Choose from R1 or a workload-specific weighting. The equal-weight summary includes R2 and R3 and cannot represent every workload.

CH01-Q07

$$P_{\mathrm{new}} = 40(0.8)^2(0.8) = 40(0.512) = 20.48\ \mathrm{W}.$$

Fixed cycles and 0.8 frequency give 1.25 times the duration:

$$t_{\mathrm{new}} = 500/0.8 = 625\ \mathrm{ms}.$$

Under the constant-power approximation,

$$E_{\mathrm{new}} = 20.48(0.625) = 12.8\ \mathrm{J}, \qquad E_{\mathrm{old}} = 40(0.5) = 20\ \mathrm{J}.$$

$$(E_{\mathrm{old}} - E_{\mathrm{new}})/E_{\mathrm{old}} = (20 - 12.8)/20 = 36\%.$$

CH01-Q08

Eliminating parser time entirely gives the optimistic bound

$$S_{\mathrm{latency,max}} = 1/(1 - 0.15) = 1/0.85 = 1.176.$$

The stated threefold parser speedup, with other times fixed, gives

$$T_{\mathrm{new}} = 0.85 + 0.15/3 = 0.90, \qquad S = 1/0.90 = 1.111.$$

Throughput depends on saturated shared resources and service stages. Reducing parser CPU work need not increase completed requests when memory bandwidth is full. Higher CPU utilization may reflect waiting or contention rather than more useful work.

Chapter 1 solutions (continued)

Measure memory-bandwidth utilization and queueing latency, then each service stage's utilization and latency. These distinguish shared-memory saturation from another serial-stage queue. Confirm equal arrival rates and input distributions as well.

Verification code appendix: performance_budget.py

```
from math import prod


def cpu_time(instruction_count: float, cpi: float, frequency_hz: float) -> float:
    if instruction_count <= 0 or cpi <= 0 or frequency_hz <= 0:
        raise ValueError("all performance inputs must be positive")
    return instruction_count * cpi / frequency_hz


def weighted_cpi(classes: list[tuple[int, float]]) -> float:
    total = sum(count for count, _ in classes)
    if total <= 0:
        raise ValueError("instruction count must be positive")
    return sum(count * cpi for count, cpi in classes) / total


def amdahl_speedup(fraction: float, local_speedup: float) -> float:
    if not 0 <= fraction <= 1 or local_speedup <= 0:
        raise ValueError("fraction must be in [0,1] and speedup must be positive")
    return 1 / ((1 - fraction) + fraction / local_speedup)


def main() -> None:
    p_time = cpu_time(3.6e8, 1.50, 2.4e9)
    q_time = cpu_time(3.0e8, 1.80, 2.8e9)
    print(f"P CPU time: {p_time * 1e3:.1f} ms")
    print(f"Q CPU time: {q_time * 1e3:.1f} ms")
    print(f"Q/P speedup: {p_time / q_time:.3f}x")
```

Chapter 1 solutions (continued)

```
    cpi = weighted_cpi(
        [
            (2_000_000_000, 1.0),
            (500_000_000, 4.0),
            (500_000_000, 3.0),
        ]
    )
    print(f"weighted CPI: {cpi:.3f}")
    assert abs(cpi - 11 / 6) < 1e-12

    speedup = amdahl_speedup(0.72, 6.0)
    print(f"Amdahl speedup: {speedup:.3f}x")
    assert abs(speedup - 2.5) < 1e-12

    ratios = [1.44, 1.00, 0.81]
    geometric_mean = prod(ratios) ** (1 / len(ratios))
    print(f"SPEC-style geometric mean: {geometric_mean:.3f}")
    assert 1.05 < geometric_mean < 1.06

    print("performance budget checks: PASS")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study